Members Excel Export: Activity Matrix + Charts

Problem

The "Membres" Excel export (`src/features/Members/utils.ts`, `downloadMembersAsExcel`) has a "DISTRIBUTION PAR TYPE D'ACTIVITÉ" section that only shows aggregate counts per activity *type* (Événements / Réunions / Formations / Assemblée). The user wants this section to instead show, per individual activity, which members attended — one column per activity, one row per member, with:

- ✓ if the member attended
- ✗ if the member did not attend
- `-` if the member was not yet a member at the time
  the width oc colums must big same thing for rows please

The export should also include several graphical charts (role distribution, activity-type participation, per-activity attendance rate, member status breakdown) so the file communicates trends visually, not just as tables.

The user also wants the per-member table itself to drop the `Rôle` and `Poste` columns when the export contains only one distinct role (e.g. downloading a single-role filter), since repeating the same role on every row is redundant — the role should instead be shown once, in the main title.

Existing code that already solves half the problem

`utils.ts` already contains a `buildActivityMatrix` function that renders exactly this ✓/✗/- matrix — but onto a *separate* worksheet ("Présences par activité"), and it's currently dead code: the only caller, `MembersPage.tsx`, never passes the `activityDetails` / `participationsWithActivityId` options it needs, even though the underlying service methods (`getAllActivitiesWithDetails`, `getParticipationsWithActivityId` in `participationService.ts`) already exist and are unused elsewhere.

Decisions (confirmed with user)

1. **Replace, don't duplicate.** The aggregate "DISTRIBUTION PAR TYPE D'ACTIVITÉ" table on the Membres sheet is replaced by the per-activity/member matrix. The separate "Présences par activité" sheet is removed — its matrix now lives inline in the Membres sheet's distribution section instead.
2. **Charts are rendered PNG images**, not native Excel chart objects (ExcelJS 4.4 cannot create editable chart objects — confirmed hard library limitation). Charts are drawn by hand on an off-screen `<canvas>` (no new npm dependency) and embedded via `worksheet.addImage`.
3. **Four charts**, placed at the bottom of the Membres sheet, 2×2 grid:
   - Role distribution (pie)
   - Activity type participation (bar): total Events/Meetings/Formations/Assembly participations
   - Per-activity attendance rate (bar): % of eligible members who attended, one bar per individual activity
   - Member status breakdown (pie): Active / Inactive / Suspended / Not yet member

4. **Single-role collapse.** When the exported members reduce to exactly one distinct role, the `Rôle` and `Poste` columns are dropped from the table entirely, and the role name is appended to the main title instead (e.g. `MEMBRES JCI HAMMAM SOUSSE – PRÉSIDENT`).

Design

1. Data wiring — `MembersPage.tsx`

In `handleDownload`, add two calls alongside the existing `Promise.all` batch:

- `participationService.getAllActivitiesWithDetails(earliestDataDate)` → `activityDetails`
- `participationService.getParticipationsWithActivityId(memberIds, earliestDataDate)` → `participationsWithActivityId`

Pass both through to `downloadMembersAsExcel(members, { ...existing, activityDetails, participationsWithActivityId })`.

These types already match `ActivityDetail[]` / `ParticipationActivityId[]` in `utils.ts` — no type changes needed there.

2. Distribution section becomes the matrix — `utils.ts`

- Delete `buildActivityMatrix` as a separate-sheet builder; delete `addDistributionTable`'s current aggregate-table body.
- New `addDistributionTable(ws, startRow, groupedMembers, activityDetails, participationsWithActivityId, periodEnd)`:
  - Same title styling as today ("DISTRIBUTION PAR TYPE D'ACTIVITÉ" — keep the French label since it's user-facing copy already in use elsewhere in the doc; can be revisited if the user wants a new title wording).
  - Header row: `Membre`, then one column per `activityDetails[i].name`.
  - One row per member (from `groupedMembers` flattened, same member set as the main table).
  - Cell logic identical to today's `buildActivityMatrix` per-cell logic: if `member.joined_at ?? member.created_at` is after `periodEnd` → `-`; else if the member is in `attendanceByActivity[activity.id]` → ✓ (green, bold); else → ✗ (red).
  - If `activityDetails` is empty or there are no members, render a single "Aucune activité dans la période sélectionnée" row (same fallback text as today), so the section degrades gracefully instead of rendering an empty table.
  - Returns the next free row index, same contract as today, so the chart section below can keep stacking.
- Call site: `downloadMembersAsExcel` already gates the old call behind `if (participationMap)`; change the gate to also require `activityDetails && participationsWithActivityId`, matching how `buildActivityMatrix` used to be gated. If either is missing, skip the section entirely (no broken partial table).

3. Chart rendering — new file `src/features/Members/chartRenderer.ts`

Two small canvas-drawing functions, framework-free:

```
export interface ChartDatum { label: string; value: number; color: string }

export function renderPieChart(data: ChartDatum[], opts?: { width?: number;
height?: number }): string // returns PNG data URL
export function renderBarChart(data: ChartDatum[], opts?: { width?: number;
height?: number }): string
```

- Uses `document.createElement('canvas')`, 2D context, draws slices/bars + a simple legend (label + value) using the existing brand palette already defined in `utils.ts` (`typeColors`, `roleColors`, etc. — reuse hex values, converting `FFRRGGBB` ARGB to `#RRGGBB` for canvas).
- Returns `canvas.toDataURL('image/png')`. ExcelJS's `workbook.addImage({ base64, extension: 'png' })` accepts a full data URL directly (verified against the installed exceljs source — it strips everything before the comma).
- No animation, no interactivity — static raster chart matching the export's existing color scheme.

4. Wiring charts into the sheet — `utils.ts`

After the existing "RÉSUMÉ DES COMITÉS" section (or after the new distribution matrix if `includeTeams` is off), add a `addChartsSection(wb, ws, startRow, {...})` that:

- Computes the four datasets:
  1. Role counts — already available via `groupedMembers`.
  2. Activity-type totals — reuse `totalEvents/totalMeetings/totalFormations/totalAssemblies` already computed in the participation summary block (lift them to be available at this scope, or recompute the same reduce — cheap, avoids threading extra params through unrelated code).
  3. Per-activity attendance rate — derived from the same `attendanceByActivity` map used by the distribution matrix: `rate = attendees / eligibleMembers * 100` per activity, where eligible members are those not showing `-` for that activity (i.e., joined on/before `periodEnd`, consistent with the matrix's own eligibility rule).
  4. Member status counts — tally `getStatus()` results (`Actif` / `Inactif` / `Suspendu` / `Pas encore membre`) while iterating members in the main loop (small addition: bump a counter object instead of only writing the cell value).
- Renders each dataset to a PNG via `chartRenderer`, registers it with `wb.addImage(...)`, and places it with `ws.addImage(imageId, { tl: { col, row }, ext: { width, height } })` in a 2×2 grid (~8 columns / ~18 rows per chart cell, tuned to roughly 400×300px charts).
- Adds a bold title cell above each chart image (reusing the section-title styling already used elsewhere).

5. Single-role column collapse — `utils.ts`

Today `downloadMembersAsExcel` builds `columnDefs` (deciding whether to include the `Poste` column) *before* computing `groupedMembers`/`sortedRoles`. That ordering has to flip, since the new rule needs to know how many distinct roles are present before the column list — and therefore the header row — can be finalized:

1. Move the `groupedMembers` reduce (role filter + exclusion of `JCI Hammam Sousse` / `jci.hs` emails) up to the top of the function, before `hidePost`/`columnDefs` are built. `sortedRoles = Object.keys(groupedMembers).sort()` moves up with it. The reduce logic itself is unchanged.
2. `const isSingleRole = sortedRoles.length === 1`.
3. `hidePost` becomes `isSingleRole || <existing isSimpleRole-based check>` — single-role mode always hides Poste, on top of the existing simple-role rule.
4. Column building:

   ```
   let columnDefs = BASE_COLUMN_DEFS;
   if (isSingleRole) {
     columnDefs = columnDefs.filter(d => d.key !== 'role');
   } else if (!hidePost) {
     // existing Poste-insertion logic, unchanged
   }
   ```

   (`Poste` is never inserted in single-role mode since that branch is skipped entirely.)
5. Title row: `titleRow.getCell(1).value = isSingleRole ? \MEMBRES JCI HAMMAM SOUSSE — ${sortedRoles[0].toUpperCase()}` : 'MEMBRES JCI HAMMAM SOUSSE'`.
6. Everything downstream (the per-role member loop, `cellValues.role`/`cellValues.post` assignment, summary sections) is unaffected: `cellValues` still sets `role`/`post` keys, they're just silently unused when absent from `columnDefs` — same pattern the code already relies on elsewhere (`columnDefs.forEach(def => cell.value = cellValues[def.key] ?? '-')`).

Non-goals

- No new npm dependency (no chart.js/recharts-to-image pipeline) — canvas is drawn by hand.
- Charts are not editable/interactive in Excel; they are pictures.
- No change to the existing member table, summary, or committee sections beyond what's needed to compute chart data.
- No change to non-Excel parts of the Members feature (UI, filters, etc.) beyond passing the two new options through `handleDownload`.

Files touched

- `src/features/Members/pages/MembersPage.tsx` — fetch + pass `activityDetails`, `participationsWithActivityId`.
- `src/features/Members/utils.ts` — replace `addDistributionTable`, delete `buildActivityMatrix`, add chart section wiring, reorder `groupedMembers`/column-def computation for single-role collapse.
- `src/features/Members/chartRenderer.ts` — new file, canvas pie/bar chart rendering to PNG data URLs.